

# Tree-Based ML — Quick Reference

*For SIH26083 Heatwave Early Warning Model*

---

## 1. Decision Trees

A decision tree predicts an outcome by asking a sequence of yes/no questions about your features, splitting the data into smaller and smaller groups until each group is (mostly) one class.

**How a split is chosen** At each step, the tree tries every possible feature and threshold (e.g. "Is WBGT > 32°C?") and picks the one that makes the resulting two groups as *pure* as possible — meaning each group is dominated by a single class rather than a 50/50 mix.

**Gini Impurity** This is the standard way to measure purity. For a group with class probabilities  $p_1, p_2, \dots$  the formula is:

$$\text{Gini} = 1 - \sum (p_i^2)$$

- Gini = 0 → perfectly pure (every example in the group is the same class)
- Gini = 0.5 → maximally mixed (for a binary problem, 50/50 split)

The tree picks the split that produces the lowest *weighted average* Gini across the two resulting groups.

**Entropy / Information Gain (alternative to Gini)** Entropy measures the same idea (disorder in a group) using a logarithmic formula instead. Information Gain = entropy before the split minus weighted entropy after. In practice, Gini and Entropy usually pick very similar splits — Gini is slightly cheaper to compute and is scikit-learn's default. You don't need both; picking Gini is fine.

**Overfitting and Pruning** Left unchecked, a tree will keep splitting until every leaf has just one example — which means it has memorized the training data and will perform badly on new data. Fixes:

- **max\_depth** — cap how many splits deep the tree can go
- **min\_samples\_leaf** — require a minimum number of examples per leaf
- **Pruning** — grow the full tree, then cut back branches that don't meaningfully improve validation accuracy

**Why this matters for your project:** a single decision tree is rarely your final model, but it's the building block for everything below.

---

## 2. Random Forest

A single tree is unstable — small changes in the data can produce a very different tree, and it overfits easily. Random Forest fixes this by building *many* trees and combining their answers.

### The two sources of randomness

1. **Bagging (Bootstrap Aggregating)** — each tree is trained on a random sample of the training data, drawn *with replacement* (so some rows repeat, some are left out).
2. **Random feature selection** — at each split, the tree is only allowed to consider a random subset of features, not all of them.

This double randomness ensures the trees make *different* mistakes rather than all making the same mistake — which is what makes averaging their answers useful.

**Voting** For classification, each tree "votes" for a class, and the forest predicts whichever class gets the most votes. For regression, it averages the trees' outputs instead.

**Why it works better than one tree:** individual trees have low bias but high variance (they overfit differently each time). Averaging many high-variance, low-bias trees cancels out the variance while keeping the low bias — a genuinely nice statistical result, not just an engineering trick.

**In practice:** `sklearn.ensemble.RandomForestClassifier(n_estimators=200, class_weight='balanced')` — start here, it's a strong and fast baseline before reaching for boosting.

---

## 3. Gradient Boosting

Random Forest builds trees *independently* and averages them. Gradient Boosting builds trees *sequentially*, where each new tree's entire job is to correct the mistakes of the trees built so far.

### The core loop

1. Start with a simple initial prediction (e.g. the average outcome).
2. Compute the *residuals* — how wrong that prediction currently is for each example.

3. Train a small decision tree to predict those residuals (not the original target — the *error*).
4. Add this new tree's predictions to the running total, scaled down by a **learning rate** (e.g. 0.1) so no single tree overcorrects.
5. Repeat steps 2–4 for many rounds (**n\_estimators**), each new tree chipping away at whatever error remains.

**Why "gradient"?** The residuals being fitted are actually the negative gradient of the loss function with respect to the current prediction — i.e., the direction that would most reduce error if you nudged your prediction that way. For classification, the loss function and residual math work out slightly differently than for regression, but the sequential "fix what's still wrong" logic is identical.

**Learning rate trade-off:** lower learning rate + more trees = usually better accuracy, but slower to train and more prone to overfitting if you add too many trees without limiting depth.

**Gradient Boosting vs Random Forest:** boosting usually wins on raw accuracy because it's *actively targeting* the model's current weaknesses, rather than just averaging independent guesses. The cost is that boosting is more sensitive to overfitting and hyperparameter choices, and trains sequentially (slower) rather than in parallel.

---

## 4. XGBoost / LightGBM

These are optimized, production-grade implementations of gradient boosting — not a different algorithm, just a faster and more careful engineering of the same idea above.

**What XGBoost adds on top of plain gradient boosting:**

- **Regularization** — penalizes overly complex trees directly in the loss function, reducing overfitting risk automatically
- **Built-in handling of missing values** — no need to impute missing weather readings by hand
- **Parallelized tree construction** — much faster training than a naive implementation
- **scale\_pos\_weight** parameter — directly addresses class imbalance (see Section 5) by upweighting the minority class during training

**LightGBM** is a similar library from Microsoft, generally faster on very large datasets because it grows trees leaf-wise instead of level-wise, but for a hackathon-scale dataset (a few years of daily weather data), XGBoost and LightGBM will perform similarly — pick whichever installs more easily in your environment.

**You do not need to implement any of this math by hand.** For this project, the practical skill is calling the library correctly:

```
from xgboost import XGBClassifier

model = XGBClassifier(

    n_estimators=200,

    learning_rate=0.1,

    max_depth=4,

    scale_pos_weight=ratio_of_negative_to_positive_class

)

model.fit(X_train, y_train)
```

---

## 5. Evaluation — Especially for Rare Events

Heatwave days are a small fraction of all days in a season. This is a **class imbalance** problem, and it breaks the usual way people check "is my model good?"

**Why accuracy lies to you** If 98% of days are "normal" and your model just predicts "normal" every single time, it scores 98% accuracy while being completely useless — it will never catch an actual heatwave. Accuracy is the wrong metric here.

**The metrics that actually matter:**

| Metric           | What it tells you                                            | Formula                         |
|------------------|--------------------------------------------------------------|---------------------------------|
| <b>Precision</b> | Of the days you flagged as heatwave, how many actually were? | $TP / (TP + FP)$                |
| <b>Recall</b>    | Of the actual heatwave days, how many did you catch?         | $TP / (TP + FN)$                |
| <b>F1-Score</b>  | Balance of precision and recall in one number                | $2 \cdot (P \cdot R) / (P + R)$ |

| Metric                  | What it tells you                                                         | Formula                  |
|-------------------------|---------------------------------------------------------------------------|--------------------------|
| <b>Confusion Matrix</b> | Raw counts of TP/FP/TN/FN<br>— the source data for everything above       | —                        |
| <b>ROC-AUC</b>          | How well the model ranks positive vs negative cases across all thresholds | area under the ROC curve |

**For a disaster early-warning system, Recall usually matters more than Precision** — missing a real heatwave (false negative) is far more costly than a false alarm (false positive), though too many false alarms cause "alert fatigue" so it's a genuine balance, not a one-sided priority.

**Handling the imbalance during training** (not just at evaluation):

- `class_weight='balanced'` in sklearn models
- `scale_pos_weight` in XGBoost (set to `count(negative) / count(positive)`)
- **SMOTE** (Synthetic Minority Over-sampling) — generates synthetic minority-class examples by interpolating between real ones, giving the model more positive examples to learn from

**Cross-validation:** instead of one train/test split, split the data into k folds (commonly 5), train on k-1 folds and validate on the remaining one, rotating through all folds. This gives a more reliable estimate of real-world performance than a single split, especially with a small number of rare positive examples where one unlucky split could badly skew your results.

---

## Quick Summary Table

| Topic             | One-line takeaway                                                      |
|-------------------|------------------------------------------------------------------------|
| Decision Tree     | Splits data to minimize Gini impurity; prone to overfitting alone      |
| Random Forest     | Many trees, randomized independently, vote together — reduces variance |
| Gradient Boosting | Trees built sequentially, each fixing the previous ones' errors        |

| Topic            | One-line takeaway                                              |
|------------------|----------------------------------------------------------------|
| XGBoost/LightGBM | Fast, regularized, production-ready gradient boosting          |
| Evaluation       | Use Precision/Recall/F1, not accuracy, when positives are rare |

# SIH26083 Implementation Notes

Project-specific addendum to Tree-Based ML - Quick Reference

## What this project adds

The original notes explain how decision trees, Random Forest, gradient boosting, and XGBoost work. This addendum maps those ideas to the locked Jaipur heatwave system.

## 1. LOYO labels: the leakage-safe target

**LOYO** means **Leave-One-Year-Out**. For each calendar date in year Y, the climatology is calculated from the same calendar window in every other year, excluding Y completely.

### For 15 May 2020:

- Use the circular calendar window 8 May through 22 May.
- Collect tmax values from 2015-2019 and 2021-2024.
- Calculate the mean, P95, and P98 from those reference values.
- Set heatwave to 1 when tmax reaches P95.
- Set severe separately when tmax reaches P98.

The persistence rule is not part of the training label. The backend checks two consecutive predicted heatwave days later.

### The label definitions are:

```
heatwave = 1 if tmax >= LOYO P95
```

```
severe = 1 if tmax >= LOYO P98
```

## 2. Features and time-aware evaluation

The model uses 17 active features:

| Feature group  | Features                                                                                         |
|----------------|--------------------------------------------------------------------------------------------------|
| Raw weather    | tmax, tmin, tmean, rh_mean, wind_speed_max, pressure_mean, solar_radiation_sum, cloud_cover_mean |
| Recent history | tmax_lag1-3 and tmin_lag1-3                                                                      |
| Climatology    | tmax_departure = tmax - LOYO normal                                                              |
| Seasonality    | doy_sin and doy_cos                                                                              |
| Reserved       | wbgt and utci are placeholders, currently inactive                                               |

Sine and cosine encode the calendar as a circle. This makes 31 December and 1 January close together instead of far apart as raw integers.

### Chronological split

Training uses 2015-2022. Testing uses 2023-2024. The data is never randomly shuffled, because a random split can let future weather patterns leak into training.

| Partition        | Rows  | Heatwave positives |
|------------------|-------|--------------------|
| Train: 2015-2022 | 2,919 | 221                |
| Test: 2023-2024  | 731   | 36                 |

### Why the first three rows disappear

Lag 1, lag 2, and lag 3 do not exist for the first three dates, so those rows are dropped after feature engineering. They are not duplicated or assigned a special label.

## 3. Imbalance handling

Heatwave days are the minority class. The pipeline follows the locked project decision:

- Random Forest uses `class_weight='balanced'`.
- XGBoost uses `scale_pos_weight = negative_count / positive_count`.
- The XGBoost weight is calculated from the training set only:  $2,698 / 221 = 12.2081$ .
- SMOTE is not used.



## 4. Models, metrics, and selection

Random Forest builds independent randomized trees and averages their votes. XGBoost builds trees sequentially, with later trees correcting the current model's errors. Both are classical tree-based models; neither is deep learning.

| Model         | Precision | Recall | F1     | CSI    |
|---------------|-----------|--------|--------|--------|
| Random Forest | 0.7742    | 0.6667 | 0.7164 | 0.5581 |
| XGBoost       | 0.8065    | 0.6944 | 0.7463 | 0.5952 |

### CSI is the locked model-selection metric:

$$\text{CSI} = \text{TP} / (\text{TP} + \text{FP} + \text{FN})$$

XGBoost wins because its CSI is higher, so `outputs/best_heatwave_model.joblib` contains the XGBoost model.

### The XGBoost test confusion matrix is:

$$\text{TN} = 689 \quad \text{FP} = 6 \quad \text{FN} = 11 \quad \text{TP} = 25$$

## 5. What happens after prediction

The FastAPI backend accepts current weather and lag values, calculates the LOYO normal and P95/P98 thresholds, constructs `tmax_departure` and cyclical date features, then calls the saved binary model.

- `heatwave_prediction` is the model's binary output.
- `severe` is true only when the prediction is heatwave and `tmax` reaches P98.
- `alert_triggered` becomes true only after two consecutive predicted heatwave days.

### Quiz memory hooks

LOYO = exclude the entire target year. P95 = heatwave. P98 = severe. CSI selects the model. The first three rows are dropped because lag features are missing.